High-Level Design (HLD) Specification

System Architecture, Trust Boundaries, Data Flows, and Scalability

1. Executive Summary & Problem Definition

DocRAG is an enterprise-grade document intelligence platform designed to eliminate hallucinations in question-answering across corporate documents (PDF, DOCX). The platform guarantees grounded answers by anchoring all generated responses strictly to source text chunks indexed with pgvector, providing clickable inline citations that reference exact page numbers and document origins.

The platform prioritizes a free-tier-first operational architecture, utilizing Supabase PostgreSQL with native pgvector, Groq Cloud free-tier LLM inference (Llama 3.3 70B Versatile), and local FastEmbed embeddings (BAAI/bge-small-en-v1.5) to deliver production-grade intelligence at zero mandatory software licensing cost.

2. High-Level Component Architecture

The DocRAG architecture is decomposed into six cleanly decoupled subsystems:

- **Client Layer (SPA):** Built with React 18, Vite, JavaScript (ES6+), and Tailwind CSS. Employs Context API for session management and Axios with automated JWT rotation.
- **API Gateway / Backend:** FastAPI application providing versioned REST endpoints under /api/v1 with Pydantic validation, CORS enforcement, and dependency injection guards.
- **Relational & Vector Data Store:** PostgreSQL 16 with pgvector extension (Supabase managed or local container). Stores users, sessions, documents, and 384-dimensional dense vectors.
- **Asynchronous Task Queue:** Celery worker brokered by Redis. Handles CPU-intensive document parsing, tokenization, and vector generation asynchronously.
- **Object & File Storage:** Modular storage abstraction providing LocalStorageProvider (development) and SupabaseStorageProvider (production cloud free tier).
- **AI & Inference Pipeline:** Groq Cloud API for ultra-low-latency Llama 3.3 generation and FastEmbed for high-throughput ONNX embeddings.

3. Data Flows & Trust Boundaries

Ingestion Flow

Client uploads file -> FastAPI authenticates user -> Validates magic bytes -> Saves file to Storage -> Inserts Document record (status='uploaded') -> Commits DB Transaction -> Dispatches Celery task -> Worker extracts pages via PyMuPDF/python-docx -> Recursive chunker creates overlapping segments -> FastEmbed computes vectors -> Atomic batch insertion into document_chunks with pgvector -> Document status updated to 'completed'.

Query Flow

Client submits query -> FastAPI authenticates -> FastEmbed computes query vector -> pgvector performs cosine distance similarity search filtered by user_id -> Top-K chunks assembled inside untrusted prompt injection boundary -> Groq synthesizes response with citations -> Persisted to messages table -> Client renders answer with interactive citations.

4. Trade-Off Analysis

Design Decision	Selected Option	Trade-Off Considered	Justification
Vector Store	PostgreSQL + pgvector	Dedicated vector DB (Pinecone, Qdrant)	Maintains ACID compliance, single transactional store, eliminates multi-db sync latency.
Embedding Engine	FastEmbed (Local ONNX)	External Cloud Embedding APIs	Zero API cost, deterministic performance, immunity to external rate limits.
LLM Generation	Groq (Llama 3.3 70B)	OpenAI GPT-4o / Claude 3.5	Generous free-tier TPM limits, sub-second TTFT (Time-To-First-Token).
Task Brokering	Celery + Redis	Kafka / RabbitMQ	Lightweight footprint, reliable task acknowledgment, native Celery retry backoff.